Linear & Logistic Regression

From line-fitting to classification — the math, the intuition, and the code

Mansouri Ahmed

Regression · Cost Function · Gradient Descent · Classification · Sigmoid · Cross-Entropy

Contents

01
Supervised Learning Recap

Regression vs Classification

02
Linear Regression

Equation · Residuals · MSE · Gradient Descent

03
Linear Regression — Python

scikit-learn implementation & example

04
Logistic Regression

Sigmoid · Softmax · Decision boundary

05
Loss Functions

MSE · Binary Cross-Entropy · Categorical Cross-Entropy

06
Logistic Regression — Python

Classification example & evaluation

Supervised Learning — Quick Recap

REGRESSION

Predict a number

House price: $350,000
Movie revenue: $200M
Temperature tomorrow: 32°C

Outcome → Continuous (any real number)

This session: predict house prices, movie box office revenue

CLASSIFICATION

Predict a category

Email: Spam / Not Spam
Tumor: Malignant / Benign
Digit: 0, 1, 2 … 9

Outcome → Discrete category (class label)

This session: spam detection, disease diagnosis

01

Linear Regression

Finding the best straight line through data

Mansouri Ahmed

What is Linear Regression?

Predict a continuous output from one or more input features using a linear equation.

Real-Life Analogy: You notice bigger houses cost more. You draw a straight line through a scatter plot of (size → price). That line IS your linear regression model — it lets you predict any house price from its size.

Input (Feature) X

The thing we know.
Example: movie budget, house size, study hours

Output (Label) Y

The thing we want to predict.
Example: box office revenue, house price, exam score

The Line $y = \beta_0 + \beta_1 x$

A straight line that best passes through all the data points. β_0 is where it starts (intercept), β_1 is the slope.

Example we'll use throughout: Predict Box Office Revenue (y) from Movie Budget (x)

The Linear Regression Equation — Step by Step

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

For simple (one feature): $\hat{y} = \beta_0 + \beta_1 \cdot x$

$\hat{y}$ — Predicted value

The model's output — estimated revenue, price, etc. (NOT the true value!)

β_0 (θ_0) — Bias / Intercept

Value of $\hat{y}$ when x=0. Like the baseline — where the line crosses the y-axis.

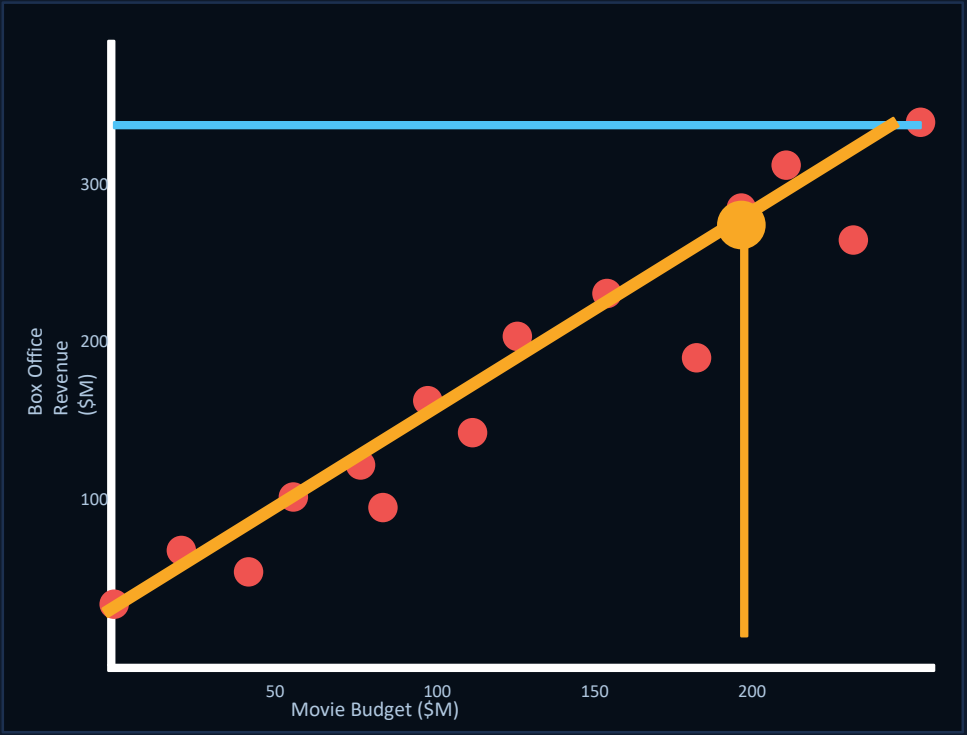
β_1 (θ_1) — Weight / Slope

How much $\hat{y}$ changes when x increases by 1 unit. Positive → up, Negative → down.

x — Feature / Input

The known data we feed in. E.g., movie budget in millions of dollars.

Worked Example — Movie Budget → Box Office Revenue



Our Model:

$$\hat{y} = 80 + 1.2 \cdot x$$

$\beta_0 = 80$

Intercept: a film with $0 budget would earn ~$80M baseline

$\beta_1 = 1.2$

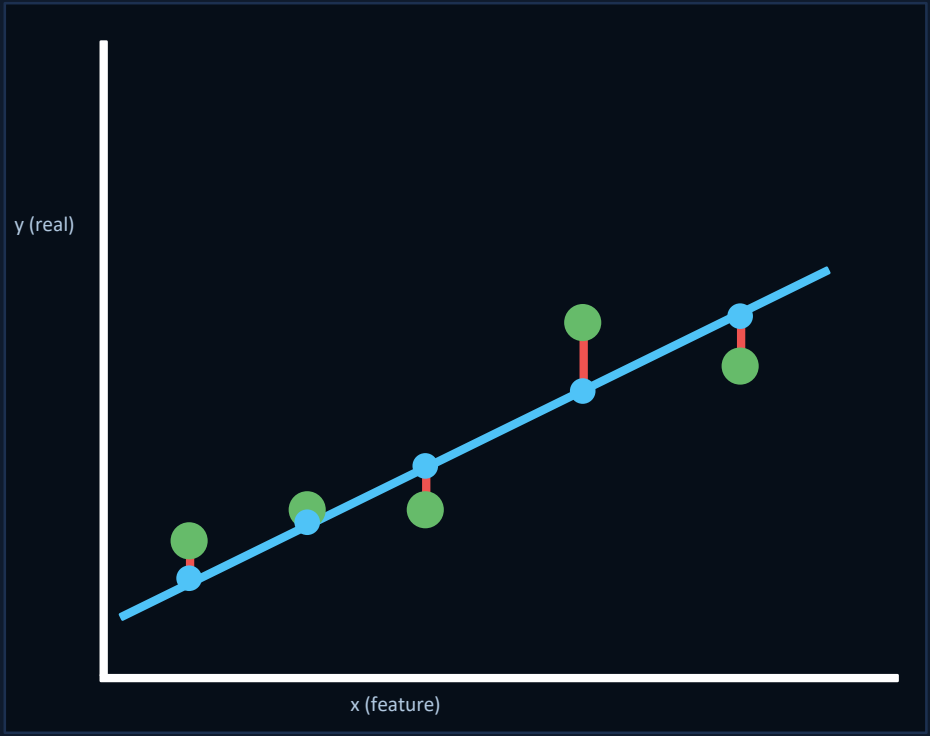
Slope: each extra $1M budget gains ~$1.2M at the box office

Prediction for Budget = $160M:

$\hat{y} = 80 + 1.2 \times 160 = 80 + 192 = \$272M$

Residuals — Measuring How Wrong We Are

Each data point has a residual = the vertical gap between the real value and our prediction.



Real value y Predicted ŷ Residual

$$e_i = y_i - \hat{y}_i$$

Residual = Real value minus Predicted value

Positive residual

Model under-predicted ($\hat{y} < y$)
Point is above the line

Negative residual

Model over-predicted ($\hat{y} > y$)
Point is below the line

Zero residual

Perfect prediction!
Point lies exactly on the line

Key insight: we want all residuals as small as possible!

Cost Function — Mean Squared Error (MSE)

We need ONE number that summarizes how wrong our model is across ALL data points.

Why square the residuals? Positive and negative errors cancel out if we just average them. Squaring makes all errors positive, and penalises large errors more harshly than small ones.

$$J(\beta_0,\beta_1) = (1/2m) \cdot \Sigma_i \, (\hat{y}_i - y_i)^2$$

$$MSE = (1/m) \cdot \Sigma_i \, (y_i - \hat{y}_i)^2$$

m	Total number of data points (training examples)
y_i	The REAL (true) value for data point i
$\hat{y}_i$	The PREDICTED value: $\hat{y}_i = \beta_0 + \beta_1 \cdot x_i$
$(\hat{y}_i - y_i)^2$	Squared residual — always positive, large errors hurt more

MSE — Numerical Example

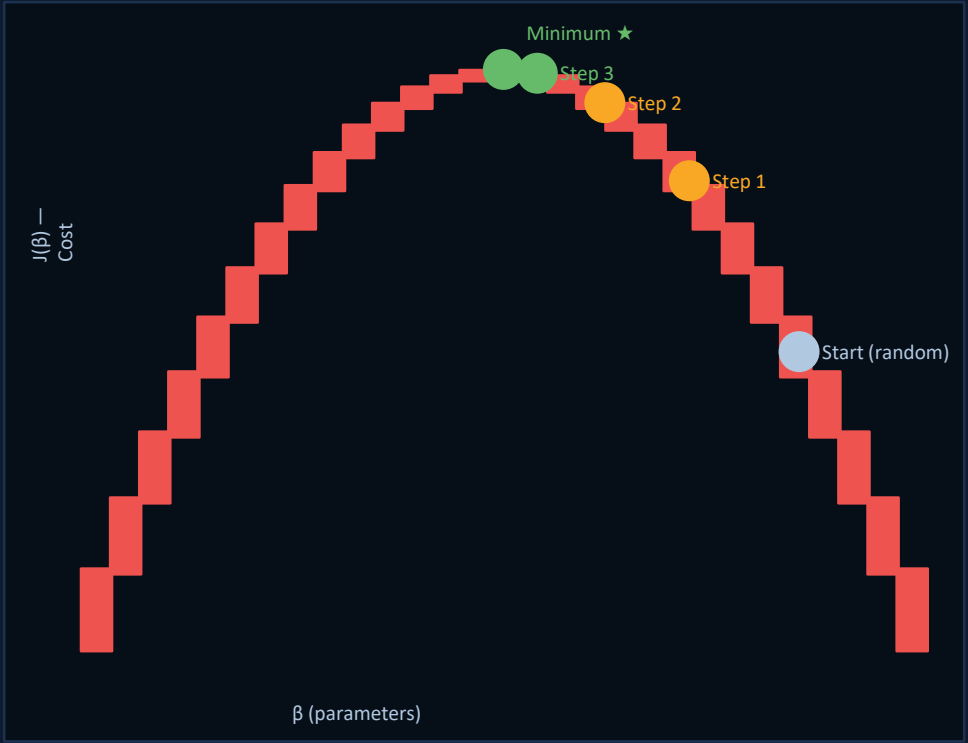
Model: $\hat{y} = 80 + 1.2x$ Let's compute the cost for 5 training examples.

#	Budget x ($M)	Revenue y ($M)	$\hat{y}$ = 80+1.2x	Residual e	e^2	
1	50	140	140	0	0	✓ perfect
2	80	145	176	-31	961	✗ over
3	120	230	224	6	36	✓ close
4	160	280	272	8	64	✓ close
5	200	310	320	-10	100	✓ close
Sum	—	—	—	—	**1161**	

MSE = $\Sigma e^2 / m$ = 1161 / 5 = 232.2 → √MSE (RMSE) = $15.2M — the average prediction error

Gradient Descent — How We Find the Best β

We can't check every possible line. Gradient Descent rolls downhill on the cost surface to find the minimum.



The Algorithm:

$$\theta_j \leftarrow \theta_j - \alpha \cdot (\partial J / \partial \theta_j)$$

Update rule — repeat until convergence

1 Compute Gradient

∂J/∂β — which direction is 'uphill' on the cost surface?

2 Step opposite direction

Subtract α × gradient from current β to move downhill

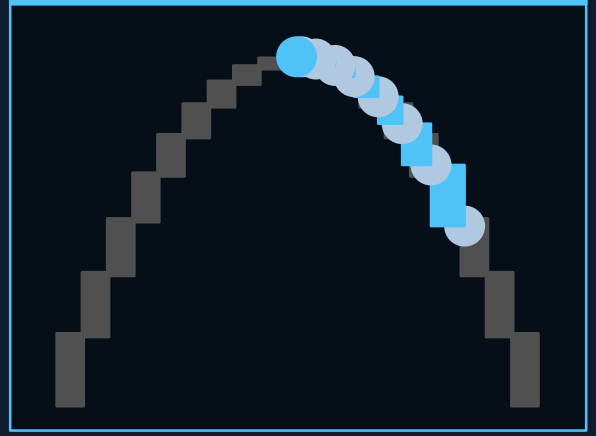
3 Repeat

Keep stepping until cost barely changes = convergence

α (learning rate): too small = slow; too large = overshoots! Best: 0.001–0.01

Learning Rate α — Getting it Right

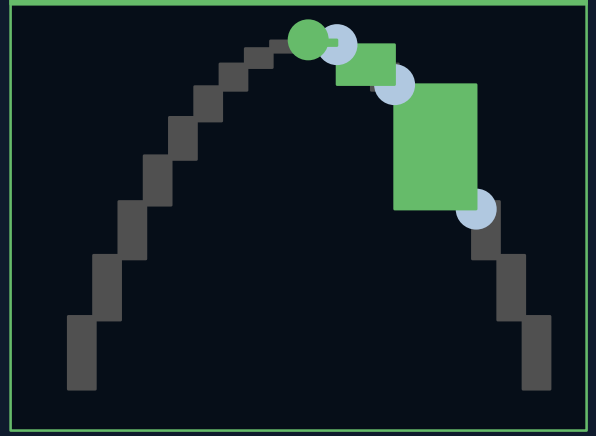
α too SMALL



Very slow convergence

Takes thousands of steps to reach the minimum. Training is painfully slow. Will eventually converge but wastes time.

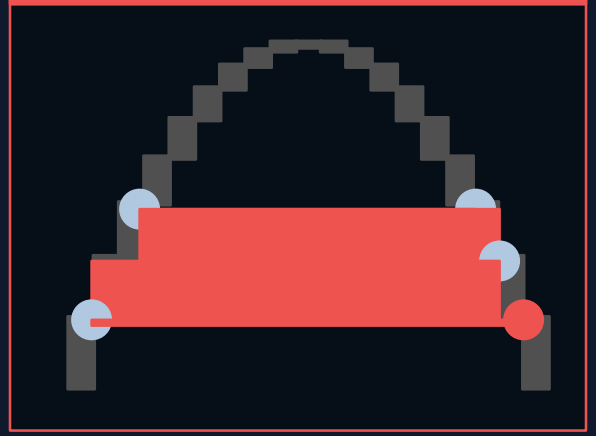
α just RIGHT



Smooth, steady convergence

Reaches the minimum in a reasonable number of steps. This is the sweet spot — try 0.001, 0.01, 0.1 and cross-validate.

α too LARGE



Diverges — never converges!

Steps so big it jumps over the minimum and oscillates wildly. Cost may actually increase. Reduce α immediately.

💡 Rule of thumb: start with α = 0.01, watch loss curve — if it decreases smoothly, good; if it spikes, halve α.

Multiple Linear Regression — More Features, More Power

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

n features, n+1 parameters

House Price Example

x_1 = Size (m²) — **$\theta_1 = +1200$**

Each extra m² adds $1,200

x_2 = #Bedrooms — **$\theta_2 = +5000$**

Each bedroom adds $5,000

x_3 = Age (yrs) — **$\theta_3 = -800$**

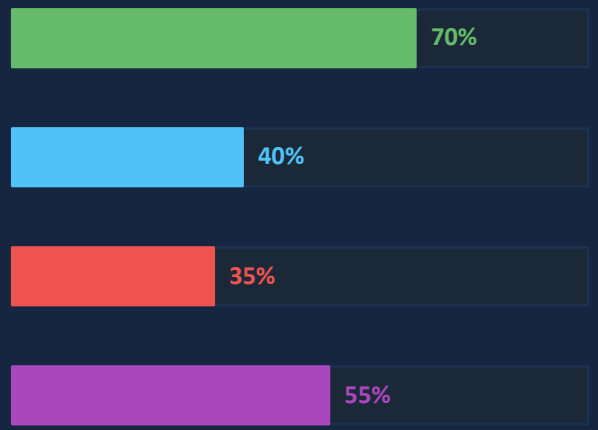
Each year reduces by $800

x_4 = Distance (km) — **$\theta_4 = -3000$**

Each km from city −$3,000

$$\hat{y} = 50000 + 1200 \cdot x_1 + 5000 \cdot x_2 - 800 \cdot x_3 - 3000 \cdot x_4$$

Feature Weights (θ) at a glance



A negative weight means that feature DECREASES the prediction.
A larger |ϑ| means that feature has MORE influence on ŷ.

Linear Regression in Python — scikit-learn

Predicting house prices with multiple features

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Features: [Size(m²), Bedrooms, Age(yr), Distance(km)]
X = np.array([[85,3,5,2],[120,4,10,5],[60,2,20,8],[200,5,2,1]])
y = np.array([280000, 350000, 150000, 600000])  # prices

# 1. Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Train model
model = LinearRegression()
model.fit(X_train, y_train)

# 3. Predict & Evaluate
y_pred = model.predict(X_test)
print(f'MSE  : {mean_squared_error(y_test, y_pred):.0f}')
print(f'RMSE : {mean_squared_error(y_test, y_pred)**0.5:.0f}')
print(f'R²   : {r2_score(y_test, y_pred):.3f}')

# 4. Inspect parameters
print('Intercept:', model.intercept_)  # β₀
print('Coefficients:', model.coef_)    # β₁…β□
```

`model.coef_`

Array of $\theta_1 \ldots \theta_{\square}$. Positive = feature increases $\hat{y}$. Negative = decreases it.

MSE / RMSE

RMSE is in the same units as y (dollars). Smaller = better. Target: RMSE < 10% of mean y.

R^2 Score

0 to 1. How much variance the model explains. $R^2 = 0.95$ means 95% of price variation is captured.

`fit_intercept`

Set to False if data is already mean-centered. Usually leave as True (default).

sklearn: no manual gradient descent needed — it uses Normal Equation or SGD internally!

02

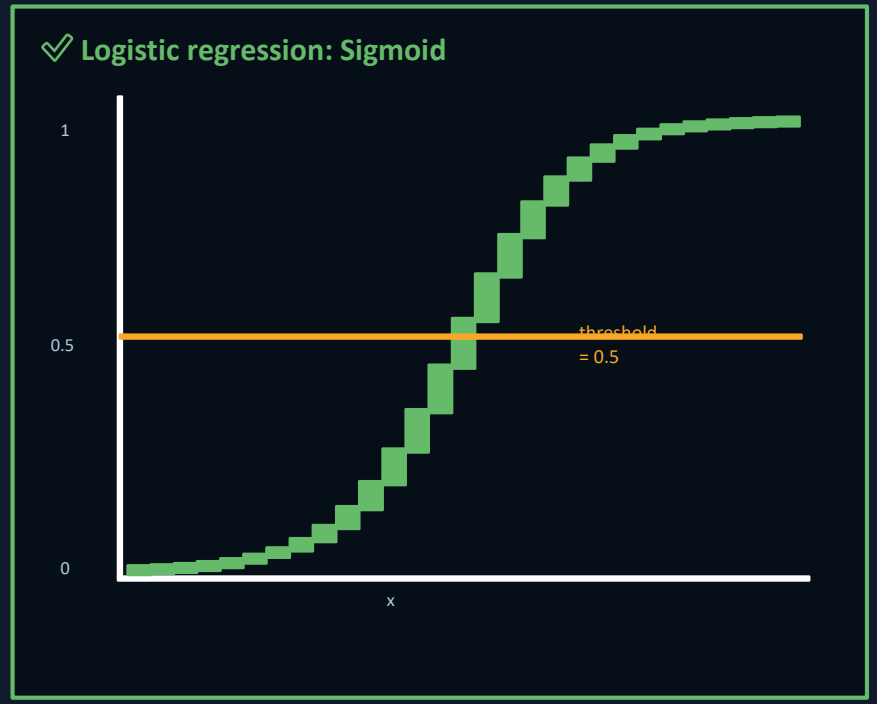
Logistic Regression

From numbers to probabilities — classifying with confidence

Mansouri Ahmed

Why Not Use Linear Regression for Classification?

Linear regression predicts any number — but class probabilities must stay between 0 and 1.



Key rule: σ(z) always outputs between 0 and 1 → interpret as PROBABILITY → if σ(z) ≥ 0.5 → class 1; else → class 0

The Sigmoid Function — Squashing to [0, 1]

$$\sigma(z) = 1 / (1 + e^{-z})$$

where $z = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$

$\sigma(z) \to 0$ *when $z \to -\infty$*

Very negative z means very low probability. Confident prediction: class 0.

$\sigma(0) = 0.5$ *at $z = 0$*

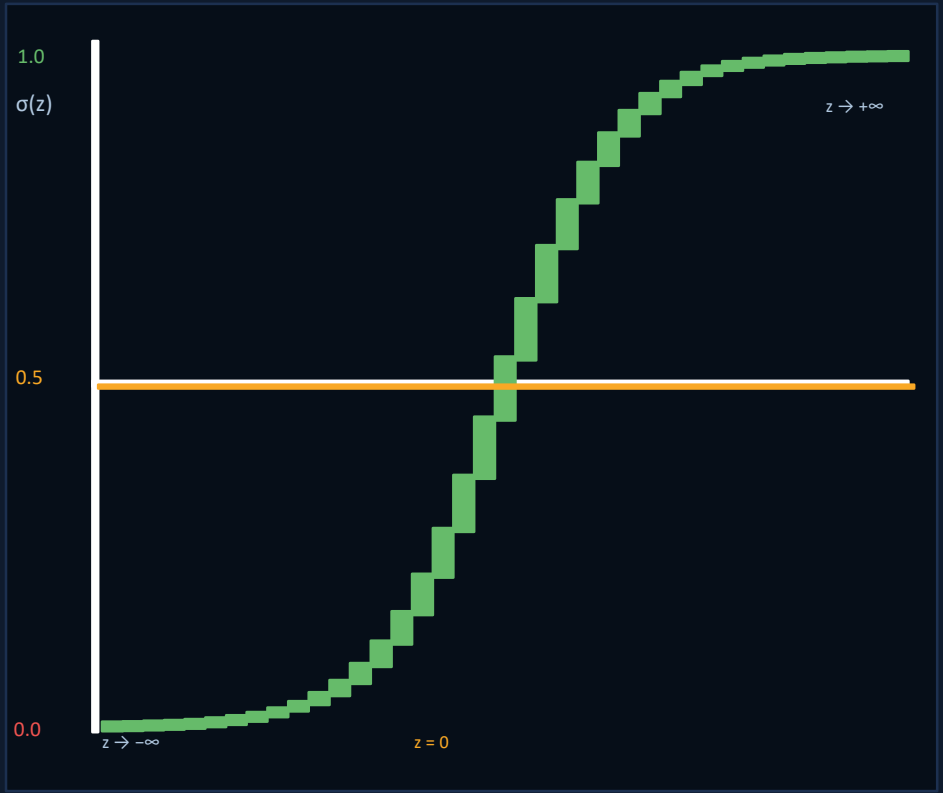
Exact boundary between classes. We're 50% uncertain — threshold point.

$\sigma(z) \to 1$ *when $z \to +\infty$*

Very positive z means very high probability. Confident prediction: class 1.

Always smooth *differentiable*

Essential for gradient descent to work — we can always compute the gradient.



Worked Example — Spam Detection

Feature: number of suspicious words in email. Predict: Spam (1) or Not Spam (0).

$$z = -3 + 0.8 \cdot x \quad \text{then} \quad P(\text{Spam}) = \sigma(z) = 1/(1+e^{-z})$$

x (words)	z = -3+0.8x	σ(z) = P(Spam)	Prediction	Verdict
2	-1.4	0.20	0	Not Spam ✅
4	0.2	0.55	1	Spam ⚠️
5	1.0	0.73	1	Spam 🚨
8	3.4	0.97	1	Spam 🚨
0	-3.0	0.05	0	Not Spam ✅

Decision rule: if σ(z) ≥ 0.5 → Predict Spam (class 1); else → Not Spam (class 0)

Softmax — Extending to Multiple Classes

Instead of Sigmoid (2 classes), use Softmax for K classes. Outputs sum to 1 — a probability distribution.

$$P(\text{class } k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j) \quad \text{where } z_k = \theta_k^T \cdot x$$

Each class k gets its own weight vector θ_k; the exponential ensures all probs are positive; dividing by sum makes them sum to 1

Example: Digit Recognition (0–9)

Digit 0	z=1.2	0.03	Digit 1	z=0.5	0.02
Digit 2	z=3.1	0.15	**Digit 3**	z=5.8	**0.72**
Digit 4	z=1.0	0.02	Digit 5	z=0.2	0.01
Digit 6	z=0.8	0.02	Digit 7	z=0.3	0.01
Digit 8	z=1.1	0.01	Digit 9	z=0.6	0.01

★ Predicted class: 3 (P=0.72)

Binary (2 classes)

→ Use Sigmoid

Spam/Not-Spam, Cancer/Benign, Pass/Fail
Output: single probability $p \in (0,1)$

Multiclass (K ≥ 3)

→ Use Softmax

Digits 0-9, Iris species, News categories
Output: K probabilities, all summing to 1

Multilabel

→ Multiple Sigmoids

Image has [cat, dog, sky] — each label predicted independently with its own Sigmoid

Loss Functions — How We Measure Classification Error

MSE is wrong for classification. We need Cross-Entropy, which penalises confident wrong predictions harshly.

Binary Cross-Entropy: $L = -(1/n) \cdot \Sigma_i \left[y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i) \right]$

True y = 1 (positive) **Pred ŷ = 0.95** **Loss = −log(0.95) = 0.051**

Correct + confident → tiny loss ✅

True y = 1 (positive) **Pred ŷ = 0.5** **Loss = −log(0.50) = 0.693**

Correct but uncertain → medium loss ⚠️

True y = 1 (positive) **Pred ŷ = 0.05** **Loss = −log(0.05) = 3.0**

Wrong + confident → HUGE loss 💥

Categorical Cross-Entropy (K classes):

$L = -(1/n) \cdot \Sigma_i \Sigma_k y_k^{(i)} \cdot \log(\hat{y}_k^{(i)})$

y_k = 1 if sample i is class k (else 0). Use with Softmax output.

Logistic Regression in Python — scikit-learn

Binary classification: predicting if a tumour is Malignant (1) or Benign (0)

```python
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler

# 1. Load dataset (569 samples, 30 features)
data = load_breast_cancer()
X, y = data.data, data.target

# 2. Scale features — critical for log. regression!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Split
X_tr, X_te, y_tr, y_te = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

# 4. Train
clf = LogisticRegression(max_iter=1000)
clf.fit(X_tr, y_tr)

# 5. Evaluate
print(classification_report(y_te, clf.predict(X_te)))
print(confusion_matrix(y_te, clf.predict(X_te)))
```

Output (typical result):

	Pred 0	Pred 1
True 0	42	1
True 1	2	69

Accuracy 96.5%	Precision 98.6%
Recall 97.2%	F1-Score 97.9%

💡 Always scale features before Logistic Regression — it uses gradient descent internally and unscaled features cause slow convergence or divergence!

Linear Regression vs Logistic Regression — Summary

	Linear Regression	Logistic Regression
Task	Predict a number (regression)	Predict a class (classification)
Output	Any real number $\hat{y} \in \mathbb{R}$	Probability $p \in (0,1)$
Activation	None (identity)	Sigmoid σ or Softmax
Loss function	MSE (Mean Squared Error)	Cross-Entropy (log loss)
Decision	$\hat{y}$ directly	if $p \geq 0.5 \rightarrow$ class 1
Metrics	RMSE, R^2 score	Accuracy, Precision, Recall, F1
Feature scaling	Recommended	Essential
Example	Predict house price ($350k)	Classify email (Spam / Not Spam)

Session Summary — Key Takeaways

Everything you need to remember from Linear & Logistic Regression

01

Linear Regression = Best Line

$\hat{y} = \beta_0 + \beta_1 x$. Finds β that minimises MSE. Used for predicting continuous values.

02

Residuals & Cost

Residual = $y - \hat{y}$. MSE = $(1/m)\Sigma(y_i - \hat{y}_i)^2$. Minimise to find the best model.

03

Gradient Descent

Iteratively adjust β in the direction of steepest descent. α controls step size — tune carefully.

04

Logistic Regression = Probability

Applies Sigmoid σ(z) to the linear output. Maps any z to (0,1). Classify at threshold 0.5.

05

Cross-Entropy Loss

MSE is wrong for classification. Use Binary CE for 2 classes, Categorical CE for K classes.

06

Always Scale Features

Logistic regression uses gradient descent internally — StandardScaler before fitting is essential.

Great work!

You now master Linear & Logistic Regression.